ВВЕДЕНИЕ 2
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ КУРСА 3
1.1. Основы искусственных нейронных сетей 3
1.2.Основы генетических алгоритмов 7
1.3. Основы нечёткой логики 13
1.4. Основы нейро-нечётких систем 23
2. ОПИСАНИЯ ЛАБОРАТОРНЫХ РАБОТ 34
Лабораторная работа №1. Изучение основ применения искусственных нейронных сетей 34
Лабораторная работа №2. Изучение основ применения генетических алгоритмов 37
Лабораторная работа № 3. Изучение основ применения нечёткой логики 39
Лабораторная работа №4. Изучение основ применения нейро-нечётких систем 41

ВВЕДЕНИЕ

В настоящее время всё большее распространение приобретают разработки в области управления и обработки информации, основанные на математических аппаратах искусственных нейронных сетей, нечёткой логики, генетических алгоритмов.

Среди задач обработки информации одной из немаловажных задач, которая может быть решена с применением указанными выше подходов, является задача анализа состояния технических систем.

Цель учебного пособия – познакомить читателя с основами применения современных подходов в задачах управления, а также анализа состояния технических систем на примере выполнения лабораторных работ.

Выполнение лабораторных работ предусмотрено в программном пакете Simintech, который за последние годы стал широко применяться во многих инженерных приложениях, что позволит получить полезный для практической работы навык.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ КУРСА

1.1. Основы искусственных нейронных сетей

1.1.1. Принципы функционирования

Под искусственными нейронными сетями (ИНС) подразумеваются вычислительные структуры, которые моделируют простые биологические процессы, обычно ассоциируемые с процессами человеческого мозга. Они представляют собой распараллеленные системы, способные к обучению и адаптации путем анализа положительных и отрицательных воздействий. Элементарным преобразователем в данных сетях является искусственный нейрон или просто нейрон, названный так по аналогии с биологическим.

Нейрон — это составная часть нейронной сети. Нейрон состоит из умножителей (синапсов), сумматора и нелинейного преобразователя.

При помощи синапсов осуществляется связь между нейронами. При этом входной сигнал нейрона умножается на число, характеризующее силу связи, — вес синапса.

Сумматор осуществляет сложение сигналов, поступающих по синаптическим связям от других нейронов, и внешних входных сигналов. Нелинейный преобразователь реализует нелинейную функцию одного аргумента — выхода сумматора. Эта функция называется функцией активации нейрона. Нейрон в целом реализует скалярную функцию векторного аргумента.

Математическая модель нейрона может быть описана следующими соотношениями

$$s = \sum_{i=1}^{n} w_i x_i + b$$

$$y = f(s)$$

В общем случае входной сигнал, весовые коэффициенты и значения смещения могут принимать действительные значения. Выход нейрона *y* определяется видом функции активации *f(s)* и может быть как действительным, так и целым. Во многих практических задачах входы,

веса и смещения могут принимать лишь некоторые фиксированные значения

Таким образом, нейрон полностью описывается своими весами w_i и передаточной функцией *f(s)*. Для заданного набора чисел (вектора) x_i в качестве входов, нейрон формирует некоторое число *y* на выходе.

Процесс функционирования ИНС зависит от значений синаптических связей. Поэтому, задав определенную структуру ИНС, необходимо найти значения всех переменных весовых коэффициентов (некоторые синаптические связи могут быть постоянными).

Этот этап называется обучением ИНС, и от того, насколько качественно он будет выполнен, зависит способность сети решать поставленные перед ней проблемы во время функционирования.

1.1.2.Обучение ИНС

Математически процесс обучения может быть описан следующим образом.

В процессе функционирования нейронная сеть формирует выходной сигнал **Y**, соответствующий входному сигналу **X**, реализуя некоторую функцию **Y = G(X)**.

Пусть решением некоторой задачи является функция **Y = F(X)**, которая представлена известными парами входных-выходных данных $(\mathbf{X^1}, \mathbf{Y^1})$, $(\mathbf{X^2}, \mathbf{Y^2})$, ..., $(\mathbf{X^N}, \boldsymbol{Y^N})$, где $\boldsymbol{Y^k = F(X^k)}$ (k = 1, 2, ...N).

Процесс обучения заключается в поиске функции **G**, близкой к **F** в смысле некоторой функции ошибки ***E***.

Если выбраны множество обучающих примеров — пар $(\mathbf{X}^k, \boldsymbol{Y}^k)$ (где k = 1,2,..., N) и способ вычисления функции ошибки **E**, то обучение нейронной сети превращается в задачу многомерной оптимизации, имеющую очень большую размерность. Т.к. функция **E** может иметь произвольный вид, обучение в общем случае — многоэкстремальная невыпуклая задача оптимизации.

Для решения этой задачи могут быть использованы следующие (итерационные) алгоритмы:

• алгоритмы локальной оптимизации с вычислением частных производных первого порядка;

• алгоритмы локальной оптимизации с вычислением частных производных первого и второго порядка;

• стохастические алгоритмы оптимизации;

• алгоритмы глобальной оптимизации.

К первой группе относятся: градиентный алгоритм (метод скорейшего спуска); методы с одномерной и двумерной оптимизацией целевой функции в направлении антиградиента; метод сопряженных

градиентов; методы, учитывающие направление антиградиента на нескольких шагах алгоритма.

Ко второй группе относятся: метод Ньютона, методы оптимизации с разреженными матрицами Гессе, квазиньютоновские методы, метод Гаусса Ньютона, метод Левенберга-Марквардта и др.

Стохастическими методами являются: поиск в случайном направлении, имитация отжига, метод Монте-Карло (численный метод статистических испытаний).

Задачи глобальной оптимизации решаются с помощью перебора значений переменных, от которых зависит целевая функция (функция ошибки **E**).

1.1.3. Переобучение и обобщение

В отсутствие полной и бесконечно большой обучающей выборки на самом деле, минимизируется не «истинная» ошибка на поверхности ошибок в заранее неизвестной модели явления, а только ошибка относительно имеющихся данных.

Сильнее всего это различие проявляется в явлении слишком близкой подгонки параметров ИНС под данные обучающей выборки, называемом переобучением. Это явление хорошо иллюстрируется на примере аппроксимации посредством полиномов.

Графики полиномов могут иметь различную форму. При этом, чем выше его степень, тем более сложной может быть эта форма. Полиномиальная кривая (модель) может быть подогнана под конкретные данные и таким образом объяснить существующую зависимость.

При этом данные могут быть зашумлены, поэтому нельзя считать, что самая лучшая модель задается кривой, которая в точности проходит через все имеющиеся точки данных. Полином низкого порядка может быть недостаточно гибким средством для аппроксимации данных, в то время как полином высокого порядка может оказаться чересчур гибким, и будет точно следовать данным, принимая при этом замысловатую форму, не имеющую никакого отношения к форме настоящей зависимости.

Нейронная сеть сталкивается с точно такой же трудностью. ИНС с большим числом весов моделируют более сложные функции и, следовательно, склонны к переобучению. ИНС с небольшим числом весов может оказаться недостаточно гибкой, чтобы смоделировать имеющуюся зависимость.

Для борьбы с переобучением используется механизм контрольной кросс-проверки, при котором часть обучающих наблюдений резервируется и в обучении не используется. Вместо этого она

используется для независимого контроля результата в процессе осуществления обучения сети.

В самом начале обучения ошибки сети на обучающем и контрольном множествах будут одинаковыми (если они существенно отличаются, то, вероятно, разбиение всех наблюдений на два множества было неоднородным). По мере того как сеть обучается, ошибка обучения будет убывать, и, пока обучение уменьшает действительную функцию ошибок, ошибка на контрольном множестве также будет убывать. Если же контрольная ошибка перестала убывать или даже стала расти, это означает, что сеть начала слишком близко аппроксимировать данные и обучение следует остановить. Этот эффект слишком точной аппроксимации в процессе обучения и называется переобучением. Если такое случилось, то обычно советуют уменьшить число скрытых элементов и/или слоев, ибо сеть является слишком мощной для данной задачи. Если же сеть, наоборот, была взята недостаточно богатой для того, чтобы моделировать имеющуюся зависимость, то переобучения, скорее всего, не произойдет, и обе ошибки — обучения и проверки — не достигнут достаточного уровня малости.

1.1.4. ИНС в задачах управления

Учитывая способность ИНС аппроксимировать функции, можно выделить следующие виды их применения в задачах управления:

1) Подражающее управление.

Нейрорегулятор обучается на примере работы обычного регулятора или человека-оператора, а потом используется взамен них.

2) Инверсное нейроуправление.

Нейрорегулятор обучается на примере поведения объекта управления и аппроксимирует функцию, описывающую зависимость значений управляющего сигнала от последующей реакции объекта управления.

3) Прогнозирующее нейроуправление.

ИНС применяется для описания объекта управления с целью определения оптимальной последовательности управляющих воздействий.

4) Многомодульное нейроуправление.

Исходная задача управления разделяется на отдельные подзадачи, которые решают отдельные модули из ИНС. Финальное решение формирует шлюзовая сеть на основе частных решений модулей-экспертов.

5) Гибридное нейроуправление.

ИНС применяется для настройки параметров ПИД-регулятора в режиме онлайн.

1.2.Основы генетических алгоритмов

1.2.1. Генетические алгоритмы и традиционные методы оптимизации

Генетический алгоритм представляет собой метод, отражающий естественную эволюцию методов решения проблем, и в первую очередь задач оптимизации. Генетические алгоритмы - это процедуры поиска, основанные на механизмах естественного отбора и наследования. В них используется эволюционный принцип выживания наиболее приспособленных особей.

Генетические алгоритмы отличаются от традиционных методов оптимизации несколькими базовыми элементами. В частности, они:

1) обрабатывают не значения параметров самой задачи, а их закодированную форму;

2) осуществляют поиск решения исходя не из единственной точки, а из их некоторой популяции;

3) используют только целевую функцию, а не ее производные либо иную дополнительную информацию,

4) применяют вероятностные, а не детерминированные правила вывода.

1.2.2. Основные понятия генетических алгоритмов

В теории генетических алгоритмов используются определения, заимствованные из генетики

Например, речь идет о *популяции особей, а в* качестве базовых понятий применяются *ген, хромосома, генотип, фенотип,* аллель.

Также используются соответствующие этим терминам определения из технического лексикона, в частности, цель, *двоичная последовательность, структура.*

Популяция - это конечное множество *особей.*

Особи, входящие в популяцию, в генетических алгоритмах представляются хромосомами с закодированными в них множествами *параметров* задачи, т.е. решений, которые иначе называются точками *в пространстве поиска.* В некоторых работах особи называются организмами.

Хромосомы (другие названия - цепочки или *кодовые последовательности)* - это упорядоченные последовательности *генов.*

Ген (также называемый *свойством, знаком* или детектором) - это атомарный элемент *генотипа,* в частности, хромосомы.

Генотип или структура - это набор хромосом данной особи. Следовательно, особями популяции могут быть генотипы либо единичные хромосомы (в довольно распространенном случае, когда генотип состоит из одной хромосомы).

Фенотип - это набор значений, соответствующих данному генотипу, т.е. *декодированная структура* или множество параметров задачи (решение, точка *пространства поиска).*

Аллель - это значение конкретного гена, также определяемое как *значение* свойства или *вариант свойства.*

Локус или п*озиция* указывает место размещения данного гена в хромосоме (цепочке). Множество позиций генов - это *локи.*

Очень важным понятием в генетически алгоритмах считается *функция приспособленности (fitness function),* иначе называемая *функцией оценки:*

Представляет меру приспособленности данной особи в популяции.

Играет важнейшую роль, поскольку позволяет оценить степень приспособленности конкретных особей в популяции и выбрать из них наиболее приспособленные (т.е. имеющие наибольшие значения функции приспособленности) в соответствии с эволюционным принципом выживания «сильнейших» (лучше всего приспособившихся).

Оказывает сильное влияние на функционирование генетических алгоритмов и должна иметь точное и корректное определение.

В задачах оптимизации функция приспособленности, как правило, оптимизируется (точнее говоря, максимизируется) и называется *целевой функцией.* В задачах минимизации целевая функции преобразуется, и проблема сводится к максимизации.

В теории управления функция приспособленности может принимать вид *функции погрешности,* а в теории игр - *стоимостной функции.* На каждой итерации генетического алгоритма приспособленность каждой особи данной популяции оценивается при помощи функции приспособленности, и на этой основе создается следующая популяция особей

Очередная популяция в генетическом алгоритме называется поколением, а к вновь создаваемой популяции особей применяется термин «новое поколение» или «поколение потомков».

1.2.3. Классический генетический алгоритм

Основной (классический) генетический алгоритм (также называемый элементарным или простым генетическим алгоритмом) состоит из следующих шагов:

1) инициализация, или выбор исходной популяции хромосом;

2) оценка приспособленности хромосом в популяции;
3) проверка условий остановки алгоритма;
4) селекция хромосом;
5) применение генетических операторов;
6) формирование новой популяции;
7) выбор «наилучшей» хромосомы,

Формирование исходной популяции заключается в случайном выборе заданного количества хромосом (особей), представляемых двоичными последовательностями фиксированной длины.

Оценивание приспособленности хромосом заключается в расчете функции приспособленности для каждой хромосомы этой популяции. Чем больше значение этой функции, тем выше «качество» хромосомы. Форма функции приспособленности зависит от характера решаемой задачи. Предполагается, что функция приспособленности всегда принимает неотрицательные значения и. кроме того, что для решения оптимизационной задачи требуется максимизировать эту функцию. Если исходная форма функции приспособленности не удовлетворяет этим условиям, то выполняется соответствующее преобразование (например, задачу минимизации функции можно легко свести к задаче максимизации).

Определение условий остановки генетического алгоритма зависит от его конкретного применения. В оптимизационных задачах, если известно максимальное (или минимальное) значение функции приспособленности, то остановка алгоритма может произойти после достижения ожидаемого оптимального значения, возможно - с заданной точностью. Остановка алгоритма также может произойти в случае, когда его выполнение не приводит к улучшению уже достигнутого значения. Алгоритм может быть остановлен по истечении определенного времени выполнения либо после выполнения заданного количества итераций. Если условие остановки выполнено, то производится переход к завершающему этапу выбора "наилучшей" хромосомы. В противном случае на следующем шаге выполняется селекция.

Селекция хромосом заключается в выборе (по рассчитанным на втором этапе значениям функции приспособленности) тех хромосом, которые будут участвовать в создании потомков для следующей популяции, т.е. для очередного поколения. Такой выбор производится согласно принципу естественного отбора, по которому наибольшие шансы на участие в создании новых особей имеют хромосомы с наибольшими значениями функции приспособленности. Существуют различные методы селекции. Наиболее популярным считается так называемый метод *рулетки (roulette wheel selection),* который свое название получил по аналогии с известной азартной игрой. Каждой хромосоме может быть сопоставлен сектор колеса рулетки, величина

которого устанавливается пропорциональной значению функции приспособленности данной хромосомы. Поэтому чем больше значение функции приспособленности, тем больше сектор на колесе рулетки.

Всё колесо рулетки соответствует сумме значений функций приспособленности всех хромосом рассматриваемой популяции. Каждой хромосоме, обозначаемой ch_i, для $i = 1, 2,..., N$ (где N обозначает численность популяции) соответствует сектор колеса $v(ch_i)$, выраженный в процентах согласно формуле

$$v(ch_i) = p_s(ch_i) \cdot 100\%,$$

$$\text{где } p_s(ch_i) = \frac{F(ch_i)}{\sum_{i=1}^{N} F(ch_i)}.$$

причем $F(ch_i)$ - значение функции приспособленности хромосомы ch_i, а $p_s(ch_i)$ - *вероятность селекции* хромосомы ch_i. Селекция хромосомы может быть представлена как результат поворота колеса рулетки, поскольку «выигравшая» (т.е. выбранная) хромосома относится к выпавшему сектору этого колеса.

Очевидно, что чем больше сектор, тем больше вероятность «победы» соответствующей хромосомы. Поэтому вероятность выбора данной хромосомы оказывается пропорциональной значению ее функции приспособленности.

Если всю окружность колеса рулетки представить в виде цифрового интервала [0, 100], то выбор хромосомы можно отождествить с выбором числа из интервала [a, b], где a и b обозначают соответственно начало и окончание фрагмента окружности, соответствующего этому сектору колеса; очевидно, что $0 \le a < b \le 100$. В этом случае выборе помощью колеса рулетки сводится к выбору числа из интервала [0,100], которое соответствует конкретной точке на окружности колеса.

В результате процесса селекции создается *родительская популяция,* также называемая *родительским пулом (mating poof) с* численностью N, равной численности текущей популяции.

Применение генетических операторов к хромосомам, отобранным с помощью селекции, приводит к формированию новой популяции потомков от созданной на предыдущем шаге родительской популяции.

В классическом генетическом алгоритме применяются два основных генетических оператора: *оператор скрещивания (crossover)* и *оператор мутации (mutation).* Однако следует отметить, *что* оператор мутации играет явно второстепенную роль по сравнению с оператором

скрещивания. Это означает, что скрещивание в классическом генетическом алгоритме производится практически всегда, тогда как мутация - достаточно редко. Вероятность скрещивания, как правило, достаточно велика (обычно 0,5 < p_c < 1), тогда как вероятность мутации устанавливается весьма малой (чаще всего 0 < p_m <0,1). Это следует из аналогии с миром живых организмов, где мутации происходят чрезвычайно редко

В генетическом алгоритме мутация хромосом может выполняться на популяции родителей перед скрещиванием либо на популяции потомков, образованных в результате скрещивания.

На первом этапе скрещивания выбираются пары хромосом из родительской популяции (родительского пула). Это временная популяция, состоящая из хромосом, отобранных в результате селекции и предназначенных для дальнейших преобразований операторами скрещиваний и мутации с целью формирования новой популяции потомков. На данном этапе хромосомы из родительской популяции объединяются в пары. Это производится случайным способом в соответствии с вероятностью скрещивания p_c, Далее для каждой пары отобранных таким образом родителей разыгрывается позиция гена (локус) *в* хромосоме, определяющая так называемую *точку скрещивания.*

Если хромосома каждого из родителей состоит из *L* генов, то очевидно, что точка скрещиваний l_k представляет собой натуральное число, меньшее *L.* Поэтому фиксация точки скрещивания сводится к случайному выбору числа из интервала *[1, L-1]*. В результате скрещивания пары родительских хромосом получается следующая пара потомков:

1) потомок, хромосома которого на позициях от 1 до l_k состоит из генов первого родителя, *а* на позициях от l_k + *1* до *L* — из генов второго родителя;

2) потомок, хромосома которого на позициях от 1 до l_k состоит из генов второго родителя, а на позициях от $!_к$ + 1 до *L* - из генов первого родителя.

Оператор мутации с вероятностью p_m, изменяет значение гена в хромосоме на противоположное (т.е. с *0* на 1 или обратно). Например, если в хромосоме [100110101010] мутации подвергается ген на позиции 7, то его значение, равное 1, изменяется на 0. что приводит к образованию хромосомы [100110001010]. Как уже упоминалось выше, вероятность мутации обычно очень мала, и именно от нее зависит, будет данный ген мутировать или нет. Вероятность p_m мутации может эмулироваться, например, случайным выбором числа из интервала [0, 1] для каждого гена и отбором для выполнения этой операции тех генов, для которых разыгранное число оказывается меньшим или равным значению p_m.

Хромосомы, полученные в результате применения генетических операторов к хромосомам временной родительской популяции, включаются в состав новой популяции. Она становится так называемой текущей популяцией для данной итерации генетического алгоритма. На каждой очередной итерации рассчитываются значения функции приспособленности для всех хромосом этой популяции, после чего проверяется условие остановки алгоритма и либо фиксируется результат в виде хромосомы с наибольшим значением функции приспособленности, либо осуществляется переход к следующему шагу генетического алгоритма, т.е. к селекции В классическом генетическом алгоритме вся предшествующая популяция хромосом замещается новой популяцией потомков, имеющей ту же численность.

Если условие остановки алгоритма выполнено, то следует вывести результат работы, т.е. представить искомое решение задачи. Лучшим решением считается хромосома с наибольшим значением функции приспособленности.

1.2.4. Генетический алгоритм в задачах управления

В задачах управления генетический алгоритм, главным образом, применяется для определения наилучших, в смысле заданного критерия качества, параметров системы. Например, для ПИД-регулятора с коэффициентами k_1, k_2, k_3 задача может быть сформулирована следующим образом.

Задаётся критерий качества:

$$\min_{k1,k2,k3} J = \int_0^T f(y,u)dt$$

Задаётся область поиска k_{min} и k_{max} и точность представления решения. В частности, в случае k_{min} = -7, k_{max} = 8 для решения задачи с точностью представления «1» потребуется использовать хромосомы длиной 12 генов. При этом первые четыре гена будут относиться к параметру k_1, следующие четыре – к параметру k_2, а последние – к k_3.

Пример возможных особей в такой задаче приведён ниже в таблице 1.1.

Таблица 1.1

Генотипы	Фенотип для k1	Фенотип для k2	Фенотип для k1
000000000000	-7	-7	-7
011100011000	0	-6	1
111111111111	8	8	8

1.3. Основы нечёткой логики

1.3.1. Введение

Работа Лотфи Заде «Fuzzy Sets», появившаяся в 1965 г. в журнале Information and Control, № 8, заложила основы моделирования интеллектуальной деятельности человека и явилась начальным толчком к развитию новой математической теории.

Л. Заде расширил классическое канторовское понятие множества, допустив, что характеристическая функция (функция принадлежности элемента множеству) может принимать любые значения в интервале [0; 1], а не только значения 0 либо 1. Такие множества были названы им нечеткими (fuzzy). Он определил также ряд операций над нечеткими множествами и предложил обобщение известных методов логического вывода modus ponens и modus tollens.

Введя затем понятие *лингвистической переменной* и допустив, что в качестве ее значений (термов) выступают нечеткие множества, Л. Заде создал аппарат для описания процессов интеллектуальной деятельности, включая нечеткость и неопределенность выражений.

Математическая теория нечетких множеств позволяет описывать множество *E,* элементы которого удовлетворяют свойству R, в виде упорядоченных пар.

$A = \{\mu_A(x)/x\}$, где $\mu_A(x)$ — *характеристическая функция,* принимающая значение 1, если x удовлетворяет свойству R, и 0 — в противном случае. Нечеткое подмножество отличается от обычного тем, что для элементов x из E нет однозначного ответа «да нет» относительно свойства R. В связи с этим нечеткое подмножество A универсального множества E определяется как множество упорядоченных пар

$A = \{\mu_A(x)/x\}$, где $\mu_A(x)$ — характеристическая *функция принадлежности* (или просто *функция принадлежности)*, принимающая значения в некотором вполне упорядоченном множестве M (например, M = **[0, 1])**

Функция принадлежности указывает степень (или уровень) принадлежности элемента x подмножеству A. Множество M называют множеством принадлежностей. Если $M = \{0, 1\}$, то нечеткое подмножество A может рассматриваться как обычное или четкое множество

1.3.2. Нечеткая и лингвистическая переменные

Понятие нечеткой и лингвистической переменных используется при описании объектов и явлений с помощью нечетких множеств.

Нечеткая переменная характеризуется тройкой (α, X, *A),* где

α - наименование переменной;

X - универсальное множество (область определения α);

A - нечеткое множество на *X*, описывающее ограничения (т.е. $\mu_A(x)$) на значения нечеткой переменной α.

Лингвистической переменной (ЛП) называется набор

<β, T, X, G, M>, где

β — наименование лингвистической переменной;

T — множество ее значений (терм-множество), представляющих собой наименования нечетких переменных, областью определения каждой из которых является множество *X*. Множество *T* называется базовым т*ерм-множеством* лингвистической переменной;

G - синтаксическая процедура, позволяющая оперировать элементами терм-множества T, в частности, генерировать новые термы (значения). Множество T ∪ G(T), где *G(T)* — множество сгенерированных термов, называется *расширенным терм-множеством* лингвистической переменной;

M - семантическая процедура, позволяющая превратить каждое новое значение лингвистической переменной, образуемое процедурой G, в нечеткую переменную, т.е. сформировать соответствующее нечеткое множество.

Замечание. Чтобы избежать большого количества символов:

1) символ β используют как для названия самой переменной, так и для всех ее значений;

2) пользуются одним и тем же символом для обозначения нечеткого множества и его названия, например терм «Высокая», являющийся значением лингвистической переменной *(*β = «цена», одновременно есть и нечеткое множество *M* («Высокая»).

Присвоение нескольких значений символам предполагает, что условия задачи позволяют разрешить возможные неопределенности.

Пример. Пусть эксперт определяет скорость движения автомобиля с помощью понятий «Малая скорость», «Средняя скорость» и «Большая скорость», при этом минимальная толщина равна 0 км/ч, а максимальная — 100 км/ч.

Формализация такого описания может быть проведена с помощью следующей лингвистической переменной <β, T, X, G, M>, где

β — скорость;

T — {«Малая скорость», «Средняя скорость», «Большая скорость»};

X — [0, 100];

G — процедура образования новых термов с помощью связок «и», «или» и модификаторов типа «очень», «не», «немного», «слегка» и т.п. Например: «Малая или средняя скорость», «Очень малая скорость» и т.д.;

M — процедура задания на *X* = [0, 100] нечетких подмножеств A_1= «Малая скорость», A_2 = «Средняя скорость», A_3 = «Большая скорость», а также нечетких множеств для термов из G(T) в соответствии с правилами трансляции нечетких связок и модификаторов «и», «или», «не», «очень», «слегка» и других операций над нечеткими множествами вида:

$$A \cap B,\ A \cup B,\ A,\ CON\ A = A^2,\ DIL\ A = A^{0,5}$$ и т.п.

1.3.3. Нечеткие выводы

Используемый в различного рода экспертных и управляющих системах механизм нечетких выводов в своей основе имеет базу знаний, формируемую специалистами предметной области в виде совокупности нечетких предикатных правил вида:

FR_1: Если *x* есть A_1, то *y* есть C_1,

FR_2: Если *x* есть A_2, то *y* есть C_2,

.....

FR_n: Если *x* есть A_n то *y* есть C_n

В указанных правилах:

x – входная переменная (имя для известных значений данных),

y – переменная вывода (имя для значения данных, которое будет вычислено);

A и *C* – термы с функциями принадлежности, определенными соответственно на *x* и *y*.

Пример подобного правила:

Если *x* – низко, то *y* – высоко.

Знание эксперта $A \rightarrow C$ отражает нечеткое причинное отношение предпосылки и заключения, поэтому его можно назвать нечетким отношением и обозначить через *R:*

$$R = A \rightarrow C,$$

Где "→" называют нечёткой импликацией.

Импликацией называется вид отношения, имеющего форму правила, используемого при рассуждениях. Различают классическую и нечеткую импликации.

Классическая импликация выражается с помощью соотношения:

Если *p* то *q*,

где *p* – утверждение, называемое антецедентом (условием), *q* – утверждение, называемое консеквентом (заключением).

Нечеткая импликация представляет собой правило, условие (антецедент) и заключение (консеквент) заданы нечеткими множествами, со своими функциями принадлежности и областями определения соответственно.

Отношение R можно рассматривать как нечеткое подмножество прямого произведения X x Y полного множества предпосылок X и заключений Y.

Таким образом, процесс получения (нечеткого) результата вывода C' с использованием данного наблюдения A' и знания $A \rightarrow C$ можно представить в виде формулы

$$C' = A' \circ R = A' \circ (A \rightarrow C),$$

где «○» - операция свертки

Как операцию композиции, так и операцию импликации в алгебре нечетких множеств можно реализовывать по-разному (при этом, естественно, будет разниться и итоговый получаемый результат), но в любом случае общий логический вывод осуществляется за четыре этапа:

1. Фаззификация
2. Формирование выводов правил
3. Объединение (агрегирование) выводов правил
4. Дефаззификация

Фаззификация

Функции принадлежности, определенные для входных переменных применяются к их фактическим значениям для определения степени истинности каждой предпосылки каждого правила.

Формирование выводов правил

Вычисленное значение истинности для предпосылок каждого правила применяется к заключениям каждого правила.

Вывод по каждому из правил формируется в результате применения логических операций и получается в виде нечетких подмножеств для каждой из выходных переменных.

В качестве операций, реализующих логический вывод, обычно используются min (минимум) или prod (умножение).

В логическом выводе на основе операции минимума функция принадлежности вывода «отсекается» по высоте, соответствующей вычисленной степени истинности предпосылки правила (нечеткая логика «И»).

В логическом выводе на основе операции умножения функция принадлежности вывода масштабируется при помощи вычисленной степени истинности предпосылки правила.

Объединение (агрегирование) выводов правил

Все нечеткие подмножества, назначенные к каждой переменной вывода (во всех правилах), объединяются вместе, чтобы формировать одно нечеткое подмножество для каждой переменной вывода.

При подобном объединении обычно используются операции максимума или суммирования.

При объединении на основе операции максимума комбинированный вывод нечеткого подмножества конструируется как поточечный максимум по всем нечетким подмножествам (нечеткая логика «ИЛИ»).

При объединении на основе операции суммирования комбинированный вывод нечеткого подмножества конструируется как поточечная сумма по всем нечетким подмножествам, назначенным переменной вывода правилами логического вывода.

Дефаззификация

На данном этапе на основе полученного объединённого нечёткого множества производится вычисление численного значения нечёткого вывода (четкое значение выходной переменной), т.е. генерация некоторой однозначно определенной численной величины *y*, отображающей своим значением объединение нечетких подмножеств выводов для соответствующих выходных переменных.

В основу синтеза дефаззификатора могут быть положены различные подходы, простейший из которых состоит в генерации такого выходного численного значения y_{max}, которое соответствует достижимому максимуму на множестве нечётких переменных в составе выходного нечёткого множества *B*, где заданные значения входных переменных активируют *r* решающих правил *FR*:

$$\mu_B(y_{max}) = \max_{1 \le i \le r}(\mu_B(y_i))$$

Таким образом, выбирается тот элемент нечеткого множества, который имеет наивысшую степень принадлежности этому множеству. Если этот элемент не является единственным, т.е. функция принадлежности имеет несколько локальных максимумов, или если имеется максимальное «плато», то выбор среди элементов, имеющих наивысшую степень принадлежности множеству, осуществляется на основе некоторого критерия. В зависимости от критерия, можно выделить несколько разновидностей данного метода:

1. Метод левого максимума – выбирается наименьшее из чисел y_1, y_2, .. ,y_n, имеющих наивысшую степень принадлежности выходному нечеткому множеству.

$$y_0 = \min(y \mid C(y) = \max_u C(u))$$

2. Метод правого максимума – выбирается наибольшее из чисел y_1, y_2, .. ,y_n, имеющих наивысшую степень принадлежности выходному нечеткому множеству.

$$y_0 = \max(y \mid C(y) = \max_u C(u))$$

Метод среднего из максимумов – в качестве искомого четкого значения y_o принимается среднее арифметическое координат локальных максимумов.

Четкое значение находится по формуле:

$$y_o = \frac{\int\limits_G y \cdot dy}{\int\limits_Y dy}$$

где G – подмножество элементов, максимизирующих *C(y)*.

Также чёткое значение может выбираться произвольно среди множества элементов, доставляющих максимум *C*:

$$y_0 \in (y \mid C(y) = \max_u C(u))$$

Высотная дефаззификация. Элементы области определения Ω, для которых значения функции принадлежности меньше, чем некоторый уровень α в расчет не принимаются, и четкое значение рассчитывается по формуле:

$$y_o = \frac{\int\limits_{Y\alpha} y \cdot \mu_C(y)dy}{\int\limits_{Y\alpha} \mu_C(y)dy}$$

где Yα — нечеткое множество α-уровня.

Нетрудно заметить, что представленные подходы к дефаззификации игнорируют дополнительную доступную информацию по конфигурации нечёткого множества. Например, во многих практических задачах выходное нечёткого множество является существенно асимметричным, и соответствующая информация может представлять определенное значение при генерации численной выходной переменной *y*.

В связи с этим, более эффективным с точки зрения полноты использования априорной информации, представляется подход на основе нечеткого центрирования. При этом y_o определяется как горизонтальная составляющая координат центра физической массы фигуры, распределенной в двухмерном пространстве, которую образует выходное нечёткое множество. Такой подход носит название центроидный метод дефаззификации и использует следующую формулу.

$$y_o = \frac{\int\limits_{Y} y \cdot \mu_C(y)dy}{\int\limits_{Y} \mu_C(y)dy}$$

1.3.4. Алгоритм Мамдани

Рассмотрим систему с базой знаний с двумя нечеткими правилами вида:

FR_1: если x_1 есть A_1 и x_2 есть B_1, тогда y есть C_1,

FR_2: если x_1 есть A_2 и x_2 есть B_2, тогда y есть C_2,

где x_1 и x_2 –входные переменные, y – переменная вывода.

A_1, A_2, B_1, B_2, C_1, C_2 – некоторые заданные термы с функциями принадлежности $\mu_{A1}(x_1), \mu_{A2}(x_1), \mu_{B1}(x_2), \mu_{B2}(x_2), \mu_{C1}(y), \mu_{C2}(y)$ соответственно. при этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений x_{10} и x_{20}.

Этапы нечёткого вывода по алгоритму Мамдани могут быть описаны следующим образом.

Фаззификация

Находятся степени истинности для предпосылок каждого правила: $\mu_{A1}(x_{10}), \mu_{A2}(x_{10}), \mu_{B1}(x_{20}), \mu_{B2}(x_{20})$ путём подстановки числовых значений входных переменных в соответствующие функции принадлежности.

Формирование выводов правил

Находятся уровни «отсечения» для предпосылок каждого из правил (с использованием операции минимум) где через «∧» обозначена операция логического минимума:

$$\alpha_1 = \mu_{A1}(x_{10}) \wedge \mu_{B1}(x_{20})$$

$$\alpha_2 = \mu_{A2}(x_{10}) \wedge \mu_{B2}(x_{20})$$

Затем находятся «усеченные» функции принадлежности заключений правил:

$$\mu'_{C1}(y) = \alpha_1 \wedge \mu_{C1}(y),$$

$$\mu'_{C2}(y) = \alpha_2 \wedge \mu_{C2}(y).$$

Агрегирование выводов правил

С использование операции максимума (далее обозначаемой как «$\wedge$») производится объединение найденных усеченных функций, что приводит к получению итогового нечеткого подмножества для переменной выхода с функцией принадлежности:

$$\mu_\Sigma(y) = \mu'_{C1}(y) \vee \mu'_{C2}(y) = (\alpha_1 \wedge \mu_{C1}(y)) \vee (\alpha_2 \wedge \mu_{C2}(y))$$

Дефаззификация

Нахождение численного значения y_0 проводится, например, центроидным методом по формуле

$$y_o = \frac{\int_Y y \cdot \mu_\Sigma(y) dy}{\int_Y \mu_\Sigma(y) dy}$$

1.3.5. Алгоритм Ларсена

Рассмотрим систему с базой знаний с двумя нечеткими правилами вида:

FR₁: если x_1 есть A_1 и x_2 есть B_1, тогда y есть C_1,

FR₂: если x_1 есть A_2 и x_2 есть B_2, тогда y есть C_2,

где x_1 и x_2 – входные переменные, y – переменная вывода.

A_1, A_2, B_1, B_2, C_1, C_2 — некоторые заданные термы с функциями принадлежности $\mu_{A1}(x_1), \mu_{A2}(x_1), \mu_{B1}(x_2), \mu_{B2}(x_2), \mu_{C1}(y), \mu_{C2}(y)$ соответственно, при этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений x_{10} и x_{20}

Этапы нечёткого вывода по алгоритму Ларсена могут быть описаны следующим образом.

Фаззификация

Находятся степени истинности для предпосылок каждого правила: $\mu_{A1}(x_{10}), \mu_{A2}(x_{10}), \mu_{B1}(x_{20}), \mu_{B2}(x_{20})$ *(аналогично алгоритму Мамдани)*.

Формирование выводов правил

Как в алгоритме Мамдани вначале находятся значения степеней истинности правил:

$$\alpha_1 = \mu_{A1}(x_{10}) \wedge \mu_{B1}(x_{20})$$

$$\alpha_2 = \mu_{A1}(x_{10}) \wedge \mu_{B2}(x_{20})$$

Однако нечеткая импликация реализуется с использованием оператора умножения (а не минимума):

$$\mu'_{C1}(y) = \alpha_1 \mu_{C1}(y),$$

$$\mu'_{C2}(y) = \alpha_2 \mu_{C2}(y).$$

Объединение выводов правил

На данном этапе находится итоговое нечеткое подмножество с функцией принадлежности с использованием операции максимума, что приводит к получению итогового нечеткого подмножества для переменной выхода с функцией принадлежности

$$\mu_\Sigma(y) = \mu'_{C1}(y) \vee \mu'_{C2}(y) = (\alpha_1 \mu_{C1}(y)) \vee (\alpha_2 \mu_{C2}(y))$$

Дефаззификация

Производится одним из рассмотренных ранее методов. Например, аналогично алгоритму Мамдани, центроидным методом.

1.3.6. Упрощенный алгоритм нечеткого вывода

Исходные правила в данном случае задаются в виде:

FR₁: если x_1 есть A_1 и x_2 есть B_1, тогда $y_1 = c_1$,

FR₂: если x_1 есть A_2 и x_2 есть B_2, тогда $y_2 = c_2$,

где x_1 и x_2 —входные переменные, y – переменная вывода.

A_1, A_2, B_1, B_2, – некоторые заданные термы с функциями принадлежности $\mu_{A1}(x_1), \mu_{A2}(x_1), \mu_{B1}(x_2), \mu_{B2}(x_2)$, c_1 и c_2 – некоторые обычные (четкие) числа, при этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений $x_{1\,0}$ и $x_{2\,0}$

Этапы нечёткого вывода по упрощённому алгоритму могут быть описаны следующим образом.

Фаззификация

Находятся степени истинности для предпосылок каждого правила: $\mu_{A1}(x_{1\,0}), \mu_{A2}(x_{1\,0}), \mu_{B1}(x_{2\,0}), \mu_{B2}(x_{2\,0})$ *(аналогично алгоритму Мамдани)*.

Формирование выводов правил

На втором этапе находятся числа, определяющие степень истинности правил:

$$\alpha_1 = \mu_{A1}(x_{1\,0}) \wedge \mu_{B1}(x_{2\,0})$$

$$\alpha_2 = \mu_{A2}(x_{1\,0}) \wedge \mu_{B2}(x_{2\,0})$$

Объединение выводов правил и дефаззификация

На данном этапе находится четкое значение выходной переменной по формуле:

$$y_0 = \frac{\alpha_1 c_1 + \alpha_2 c_2}{\alpha_1 + \alpha_2}$$

В общем случае для n правил значение будет находиться следующим образом:

$$y_0 = \frac{\sum_{i=1}^{n} \alpha_i c_i}{\sum_{i=1}^{n} \alpha_i}$$

1.3.7. Алгоритм Цукамото

Рассмотрим систему с базой знаний с двумя нечеткими правилами вида:

FR_1: если x_1 есть A_1 и x_2 есть B_1, тогда y есть C_1,

FR_2: если x_1 есть A_2 и x_2 есть B_2, тогда y есть C_2,

где x_1 и x_2 – входные переменные, y – переменная вывода.

A_1, A_2, B_1, B_2, C_1, C_2 — некоторые заданные термы с функциями принадлежности $\mu_{A1}(x_1), \mu_{A2}(x_1), \mu_{B1}(x_2), \mu_{B2}(x_2), \mu_{C1}(y), \mu_{C2}(y)$ соответственно. При этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений x_{10} и x_{20}.

В алгоритме Цукамото функции $\mu_{C1}(y), \mu_{C2}(y)$ задаются монотонными.

Этапы нечёткого вывода по упрощённому алгоритму могут быть описаны следующим образом.

Фаззификация

Находятся степени истинности для предпосылок каждого правила $\mu_{A1}(x_{10}), \mu_{A2}(x_{10}), \mu_{B1}(x_{20}), \mu_{B2}(x_{20})$ путём подстановки числовых значений входных переменных в соответствующие функции принадлежности.

Формирование выводов правил

На данном этапе сначала находятся (как в алгоритме Мамдани) уровни «отсечения» α_1 и α_2, а затем — посредством решения уравнений $\alpha_1 = \mu_{C1}(y)$, $\alpha_2 = \mu_{C2}(y)$ – четкие значения (y_{C1} и y_{C2}) для каждого из исходных правил.

Объединение выводов правил и дефаззификация

Чёткое значение переменной вывода определяется (как взвешенное среднее y_1 и y_2)

$$y_0 = \frac{\alpha_1 y_{C1} + \alpha_2 y_{C2}}{\alpha_1 + \alpha_2}$$

В общем случае выходное численное значение (дискретный вариант центроидного метода) находится по формуле:

$$y_0 = \frac{\sum_{i=1}^{n} \alpha_i y_{Ci}}{\sum_{i=1}^{n} \alpha_i}$$

1.3.8. Алгоритм Сугено-Такаги

Рассмотрим систему с базой знаний с двумя нечеткими правилами. Особенностью алгоритма Сугено-Такаги является следующий вид правил

FR_1: если x_1 есть A_1 и x_2 есть B_1, тогда $y_{C1} = a_1x + b_1y$

FR_2: если x_1 есть A_2 и x_2 есть B_2, тогда $y_{C2} = a_2x + b_2y$

где x_1 и x_2 –входные переменные, y –переменная вывода.

A_1, A_2, B_1, B_2, C_1, C_2 — некоторые заданные термы с функциями принадлежности. При этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений x_{10} и x_{20}.

Математически алгоритм может быть описан следующим образом:

Фаззификация

Находятся степени истинности для предпосылок каждого правила: $A_1(x_0)$, $A_2(x_0)$, $B_1(y_0)$, $B_2(y_0)$ (аналогично алгоритму Мамдани).

Формирование выводов правил

Находятся степени истинности для предпосылок каждого из правил (с использованием операции минимум или умножения)

$$\alpha_1 = \mu_{A1}(x_{10}) \wedge \mu_{B1}(x_{20})$$

$$\alpha_2 = \mu_{A2}(x_{10}) \wedge \mu_{B2}(x_{20})$$

Также для заданных входных значений x_{10} и x_{20} вычисляются индивидуальные выходы правил:

$$y_{C1}^* = a_1 x_{10} + b_1 x_{20}$$

$$y_{C2}^* = a_2 x_{10} + b_2 x_{20}$$

Объединение выводов правил и дефаззификация

Итоговое значение переменной вывода определяется (как взвешенное среднее y_{C1}^* и y_{C2}^*)

$$y_0 = \frac{\alpha_1 y_{C1}^* + \alpha_2 y_{C2}^*}{\alpha_1 + \alpha_2}$$

1.2.4. Нечёткая логика в задачах управления

Применение нечёткой логики в задачах управления основано на способности нечётких систем формировать выходную величину для заданных входных параметров, основываясь на заложенной базе функций принадлежности и решающих правил.

В зависимости от имеющихся исходных данных может применяться один из рассмотренных выше алгоритмов выводы. Если необходимо построить систему на основе неформализованных знаний экспертов, наиболее подходящими являются алгоритмы Мамдани или Ларсена. В случае возможности более чёткого формулирования знаний экспертов (например, известны законы управления для разных ситуаций), то более подходящими представляются алгоритмы Цукамото или Сугено-Такаги.

1.4. Основы нейро-нечётких систем

1.4.1. Введение

Нечёткие системы обладают определёнными достоинствами. В условиях недостатка формализованных данных за счёт использования осмысленных лингвистически заданных терминов позволяют достаточно просто интерпретировать получаемые результаты. Тем не менее, они обладают низкой гибкостью по отношению к изменениям в исследуемой системе и слабо приспособлены к самообучению, т.е. к автоматической подстройке параметров функций принадлежности на основе вновь получаемой информации.

Вместе с тем, с такой задачей могут успешно справляться искусственные нейронные сети, которые в отличие от систем нечёткого вывода могут настраивать свои параметры (весовые коэффициенты) на основе выборки данных без привлечения эксперта. Однако, для корректной настройки им требуется информативная выборка, и в случае недостатка информации, экспертные оценки не могут быть напрямую учтены при настройке. Также реализуемые ими функции часто не поддаются понятной интерпретации, что может вызвать недоверие к получаемому результату.

Предлагается комбинировать аппараты нечёткой логики и ИНС, чтобы объединить преимущества каждого из них, и при этом компенсировать их недостатки. Самым эффективным способом сочетания нечёткой логики и ИНС являются нейро-нечеткие системы, которые с одной стороны, могут рассматриваться как системы нечёткого вывода (а значит, понятно интерпретировать получаемые результаты), а с другой – как ИНС, состоящими из нейронов специального вида (а значит, могут обучаться). Таким образом, на первоначальном этапе в случае отсутствия достаточной информации такую систему можно рассматривать и конструировать как нечёткую систему на основе экспертных оценок, а в процессе накопления информации донастраивать методами искусственных нейронных сетей.

1.4.2. Структура нейро-нечётких систем

Как уже было сказано ранее, для максимизации достоинств обоих подходов в теории нейро-нечётких систем предложено рассматривать нечёткую систему типа Сугено в виде эквивалентной ей ИНС специального вида, которая изображена на рис.1.1.

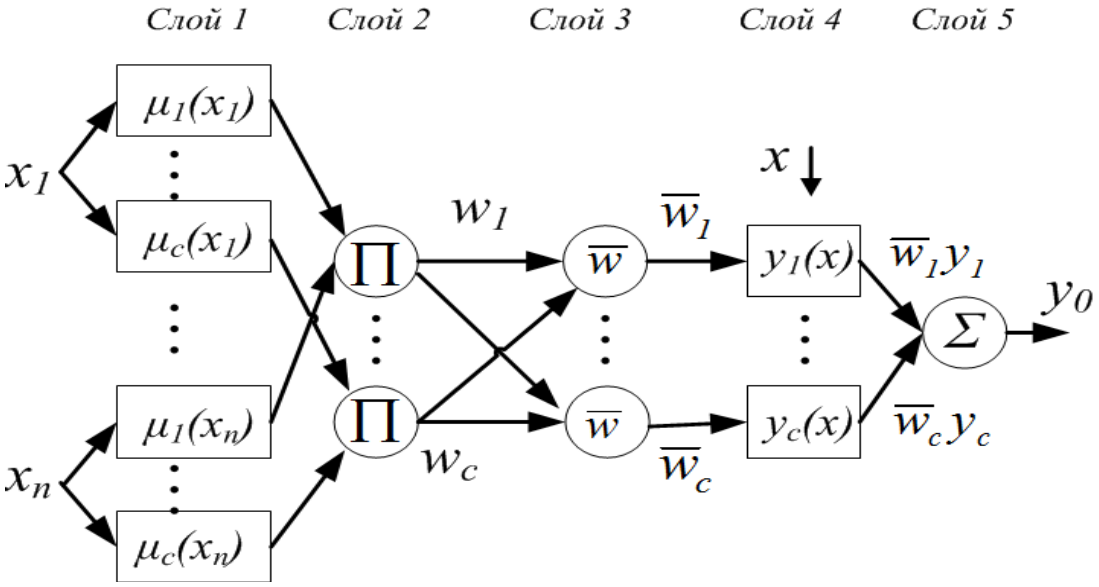


Рис.1.1. Структура нейро-нечёткой системы

Слой 1 . Содержит настраиваемые нейроны. Входами являются входные лингвистические переменные модели. Функция активации каждого из нейронов является функцией принадлежности терма лингвистической переменной.

$$\mu_i(x_j) = \mathbf{e}^{-\alpha\left(x_j - x_j^{(i)*}\right)^2}$$

Слой 2 . Является неадаптивным слоем. Выполняет операцию умножения входных сигналов. Выходами слоя являются степени активации каждого правила

$$w_i = \prod_{j=1}^{n} \mu_j(x_j), 1 \le i \le c$$

Слой 3 . Состоит из неадаптивных узлов, в которых вычисляются степени активации (веса) *i*-го правила относительно степеней активации правил – средневзвешенную степень активации правила

$$\overline{w}_i = \frac{w_i}{w_1 + w_2 + \ldots w_c}$$

Слой 4 состоит из адаптивных узлов, передаточная функция которых представляет собой полином вида $y_i(x) = g_{1,i}x_1 + \ldots + g_{n,i}x_n + h_i$ (аналогично алгоритму Сугено).

По сути, в **узлах** данного слоя рассчитывается заключение каждого из правил, которое умножается на средневзвешенную степень активации $\overline{w}_i$ соответствующего правила

$$\overline{w}_i y_i = \overline{w}_i (g_{1,i}x_1 + \ldots g_{n,i}x_n + h_i), \; i=1..c$$

Слой 5 состоит из единственного неадаптивного узла, который вычисляет конечное значение выходной переменной сети как сумму всех взвешенных, с учётом соответствующей степени активации, заключений каждого из правил

$$y_0 = \sum_{i=1}^{c} \overline{w}_i y_i$$

При этом, с точки зрения нечётких систем, этапы функционирования подобной системы аналогичны алгоритму Сугено-Такаги.

Рассмотрим систему с базой знаний с двумя нечеткими правилами.

FR_1: если $x1$ есть A_1 и $x2$ есть B_1, тогда $y_1 = g_{1,1}x1 + g_{2,1}x2 + h_1$

FR_2: если $x1$ есть A_2 и $x2$ есть B_2, тогда $y_2 = g_{1,2}x1 + g_{2,2}x2 + h_2$

где x_1 и x_2 – входные переменные, y – переменная вывода.

A_1, A_2, B_1, B_2, — некоторые заданные термы с функциями принадлежности, при этом четкое значение y_0 необходимо определить на основе приведенной информации и четких значений $x_{1\,0}$ и $x_{2\,0}$

Математически алгоритм может быть описан следующим образом:

Фаззификация

Находятся степени истинности для предпосылок каждого правила: $A_1(x_0)$, $A_2(x_0)$, $B_1(y_0)$, $B_2(y_0)$ (аналогично алгоритму Мамдани)

Формирование выводов правил

Находятся степени истинности для предпосылок каждого из правил (с использованием операции минимум или умножения)

$$w_1 = \mu_{A1}(x_{1\,0}) \wedge \mu_{B1}(x_{2\,0})$$

$$w_2 = \mu_{A2}(x_{1\,0}) \wedge \mu_{B2}(x_{2\,0})$$

Также для заданных входных значений x_{10} и x_{20} *вычисляются* индивидуальные выходы правил

$$y_1^*(x_{1\,0}, x_{2\,0}) = g_{1,1}x_1 + g_{2,1}x_2 + h_1$$

$$y_2^*(x_{1\,0}, x_{2\,0}) = g_{1,2}x_1 + g_{2,2}x_2 + h_2$$

Объединение выводов правил и дефаззификация

Чёткое значение переменной вывода определяется (как взвешенное среднее y_1^* и y_2^*)

$$y_0 = \frac{w_1 y_1^* + w_2 y_2^*}{w_1 + w_2}$$

С учётом обозначения $\overline{w}_1 = \frac{w_1}{w_1 + w_2}$, $\overline{w}_2 = \frac{w_2}{w_1 + w_2}$, получим

$$y_0 = \frac{w_1 y_1^* + w_2 y_2^*}{w_1 + w_2} = \overline{w}_1 y_1^* + \overline{w}_2 y_2^*,$$

тем самым показав эквивалентность рассмотрения подобных систем, как с точки зрения нечёткой логики, так и искусственных нейронных сетей.

1.4.3. Построение нейро-нечёткой системы

Построение нейро-нечётких систем можно производить как на основе экспертных суждений (аналогично нечётким моделям), так и на основе имеющейся выборки данных (как в случае с ИНС). Встречаются и гибридные подходы для решения указанной задачи, например, когда первоначальная структура задаётся экспертами, а дальнейшая настройка осуществляется методами обучения ИНС.

Рассмотрим более подробно методы настройки ННС на основе выборки данных.

Учитывая, что нейро-нечёткая система представляет собой ИНС специального вида с фиксированным и определённым количеством слоёв, методы их настройки будут иметь некоторые особенности.

Создание таких систем можно разделить на два основных этапа: непосредственно их построение (этап структурной настройки), и их обучение (этап параметрической настройки).

На первом этапе настраивается структура системы – определяется количество решающих правил, термов входных переменных. При этом могут определяться или задаваться начальные значения параметров функций принадлежности термов входных переменных, а также выражений в заключениях правил.

На втором этапе производится настройка значений параметров функций принадлежности термов входных переменных, а также выражений в заключениях правил на основе выборки данных.

Рассмотрим подробнее первый этап.

Введём в рассмотрение выборку данных $Z=\{Z_1, Z_2 \ldots Z_n\}$, объёмом n в M-мерном пространстве, где первые L-координат Z соответствуют вектору X входных переменных разрабатываемой модели, а оставшиеся M-L – вектору Y выходных переменных, т.е. $Z_i = (X_i, Y_i)$, $i = 1..n$.

Рассмотрим следующие способы построения нейро-нечёткой системы

- Метод решётки
- Метод субтрактивной кластеризации

Метод решётки

Пространство выборки Z разделяется на заданное количество равных областей (см. рис. 1.2)

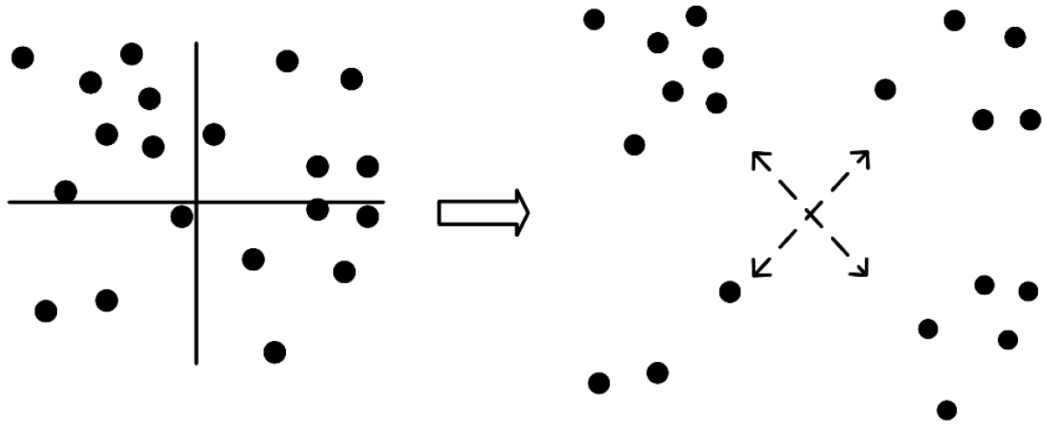


Рис.1.2. Иллюстрация процедуры разделения данных по методу решётки

Решающие правила определяются на основании проведённого разделения.

Функции принадлежности задаются из стандартного набора, максимум функций соответствует центрам выделенных областей

Метод субтрактивной кластеризации

Метод основан на разбиении имеющейся выборки данных *Z* на кластеры. Идея метода заключается в том, что каждый из кластеров, характеризующийся определёнными значениями, описывает определённое состояние исследуемой системы.

Решающие правила определяются на основании проведённого разделения. При этом максимум функции принадлежности соответствует центру кластера. Выделение кластеров производится на основе расчёта потенциалов точек.

Согласно этому методу, сначала для каждой *i*-той точки выборки *Z* рассчитывается значение потенциала P_i , $i = 1..n$:

$$P_i = \sum_{j=1}^{n} \mathrm{e}^{-\alpha \left\| Z_i - Z_j \right\|^2},$$

где $\alpha = \frac{4}{r_a^2}$ является настраиваемым параметром, определяющим область, охватываемую каждым кластером. Константа r_a – это радиус, определяющий область соседства. Таким образом, потенциал каждой точки данных P_i - это функция её расстояний до других точек данных.

Точка с большим количеством соседей будет иметь высокое значение потенциала. Точки данных вне радиуса r_a оказывают небольшое влияние на значение потенциала. После расчёта потенциала каждой из точек выбираем точку с наиболее высоким потенциалом в качестве центра первого кластера. Пусть Z_1^* – координаты центра первого

кластера, а P_1^* - значение его потенциала. Обычно, точка с максимальным значением потенциала окружена несколькими точками-соседями также с достаточно высокими потенциалами. Поэтому назначение центром следующего кластера точки с максимальным потенциалом среди оставшихся привело бы к выделению большого числа близко расположенных центров кластеров. Чтобы выбрать следующий центр кластера необходимо вначале исключить влияние только что найденного кластера. Поэтому мы корректируем значение потенциала каждой точки по следующей формуле

$$P_i \Leftarrow P_i - P_1^* e^{-\beta \left\| Z_i - Z_1^* \right\|^2}$$

где $\beta = \frac{4}{r_b^2}$ а r_b – положительная константа. Таким образом, мы вычитаем из потенциала каждой точки функцию расстояния от центра первого кластера. Таким образом, точки вблизи центра первого кластера будут иметь значительно пониженный потенциал и поэтому не будут выбраны в качестве центра следующего кластера.

Константа r_b определяет область соседних точек, у которых значительно уменьшится потенциал. Для того чтобы избежать близко расположенных центров кластеров необходимо, чтобы $r_b > r_a$; хорошим выбором является $r_b = 1.5 r_a$, т.к. такое их соотношение позволяет минимизировать количество выделяемых кластеров при сохранении точности.

Когда потенциал всех точек был изменён, точка с наивысшим потенциалом выбирается в качестве центра второго кластера. В общем случае для центра *k*-того кластера формула будет следующей:

$$P_i \Leftarrow P_i - P_k^* e^{-\beta \left\| Z_i - Z_k^* \right\|^2}$$

Где Z_k^* – положение центра *k*-того кластера, P_k^* значение его потенциала. Процедура пересчета потенциалов и выделения центров кластеров продолжается до тех пор, пока максимальное значение потенциала превышает некоторый порог. Если на *k*-том шаге $P_k^* > \bar{\varepsilon} P_1^*$, то мы принимаем Z_k^* в качестве центра кластера и продолжаем дальше. Если $P_k^* < \underline{\varepsilon} P_1^*$ то отклоняем Z_k^* и прекращаем итерационный процесс. Если не выполняются оба вышеприведённых неравенства, то проверяем, выполняется ли неравенство:

$$\frac{d_{min}}{r_a} + \frac{P_k^*}{P_1^*} \geq 1$$

где d_{min} – наикратчайшее расстояние между Z_k^* и центром кластера, найденного на предыдущем шаге. Если выполняется, то принимаем Z_k^* в

качестве центра нового кластера и продолжаем поиск. В противном случае отклоняем точку Z_k^* и устанавливаем её потенциал равным 0, выбираем точку со следующим по величине потенциалом и повторяем проверку.

Здесь $\bar{\varepsilon}$ задаёт минимальное значение потенциала, при котором мы определённо считаем точку данных центром кластера; $\underline{\varepsilon}$ определяет порог, ниже которого мы определённо отбрасываем точку. Общая методика по выбору порогов отсутствует, вместе с тем, в литературе на основе некоторых практических соображений $\bar{\varepsilon} = 0.5$ и $\underline{\varepsilon} = 0.15$ приводятся рекомендуемые значения. Неравенство отражает проверку факта достижения компромисса между текущей величиной потенциала и расстоянием от уже определённых центров кластеров. В результате получим множество центров кластеров $Z^* = \{Z_1^*, Z_2^* .. Z_c^*\}$.

Рассмотрим координаты центров выделенных кластеров. Разложим координаты каждого из центров Z_i^* на два составляющих вектора $X_i^* = (x_1^{(i)*} .. x_L^{(i)*})$ и $Y_i^* = (y_1^{(i)*} .. y_{(M-L)}^{(i)*})$, $i \in 1..c$, где X_i^* состоит из первых L координат Z_i^* (т.е. из координат центров кластеров в пространстве входов), а Y_i^* – из $M - L$ координат (т.е. из координат центров кластеров в пространстве выходов). В результате получили c соответствий, в которых для значений переменных на входе $X = X_i^*$ выходные переменные будут принимать значение $Y = Y_i^*$. Эти соответствия можно рассматривать в качестве решающих правил и использовать следующей форме:

«Если x_1 это $x_1^{(i)*}$ и x_2 это $x_2^{(i)*}$…то y_1 это $y_1^{(i)}$ и y_2 это $y_2^{(i)*}$...».

1.4.5. Определение начальных параметров функций принадлежности термов входных и выходных переменных на основе потенциалов выделенных кластеров

В предыдущем пункте показано получение термов и базы решающих правил из выборки данных. На следующем необходимо задать функции принадлежности для термов лингвистических переменных. Опираясь на существующие разработки в данной области, предлагается степень принадлежности к терму определять через введённое ранее понятие потенциала точки.

Если выражение потенциала записать для X из выборки Z, относительно X_i^* из Z_i^*, то можно определить степень принадлежности X к i-тому кластеру как

$$w_i(X) = \mathrm{e}^{-\alpha \left\| X - X_i^* \right\|^2}, \; i=1..c.$$

Указанное выражение определяет степень активации решающего правила, определяемого координатами центра кластера Z_i*. Соответственно, рассчитать значение выходного вектора Y можно как средневзвешенное значение выходов всех правил:

$$Y = \frac{\sum_{i=1}^{c} w_i Y_i^*}{\sum_{i=1}^{c} w_i}$$

Для того чтобы получить выражения функций принадлежностей для термов каждой из входных переменных, т.е. для компонент вектора $X = (x_1,.,x_L)$, перепишем в скалярной форме:

$$\mu_i(x_j) = \mathbf{e}^{-\alpha\left(x_j - x_j^{(i)*}\right)^2}$$

Полученное выражение соответствует функции принадлежности Гауссового типа для i-того терма, переменной x_j. Здесь $x_j^{(i)*}$ – j-ая координата X_i^* (т.е. центра i-того кластера), $i=1..c$, $j = 1..L$. В результате на основе выборки данных с использованием понятия потенциала кластера произведён переход от набора кластеров к системе нечёткого вывода. При этом значение потенциала точки будет соответствовать степени активации решающего правила, расчёт потенциалов по каждой координате точки – фаззификации, а расчёт выходного значения как средневзвешенной суммы центров всех кластеров – дефаззификации, что соответствует нечёткой модели Сугено-Такаги.

1.4.6. Определение начальных параметров функций принадлежности термов выходной переменной

На следующем этапе необходимо определить функции принадлежности для выходной переменной. Алгоритмом Сугено-Такаги для лучшей точности нечёткого вывода предусмотрено задание выхода каждого из правил не в виде константы, а в виде линейной функции от входных переменных:

$$Y_i^* = G_i X^* + h_i$$

Здесь Gi – это матрица размерностью (M – L) x L, а h_i – вектор-столбец с $M - L$ элементами, i = 1..c. В результате получится модель нечёткого вывода типа Сугено-Такаги 1-ого порядка, где консеквентом каждого правила является линейная комбинация входных переменных. Такая модель позволяет описывать поведение сложных систем при небольшом объёме базы решающих правил. Настройка уравнений может осуществляться на основе имеющейся выборки с данными с помощью

метода наименьших квадратов (МНК). Для этого введём следующий параметр:

$$\rho_i = \frac{w_i}{\sum_{j=1}^{c} w_i}.$$

Тогда можно получить следующее уравнение:

$$Y = \sum_{i=1}^{c} \rho_i Y_i^* = \sum_{i=1}^{c} \rho_i \left(G_i X + h_i \right)$$

Приведя которое к матричному виду, получаем:

$$\begin{bmatrix} \rho_1 X_1^T & \rho_1 & \cdots & \rho_c X_1^T & \rho_c \\ \vdots & \vdots & & \vdots & \vdots \\ \rho_1 X_n^T & \rho_1 & \cdots & \rho_c X_n^T & \rho_c \end{bmatrix} \begin{bmatrix} G_1^T \\ h_1^T \\ \vdots \\ G_c^T \\ h_c^T \end{bmatrix} = \begin{bmatrix} Y_1^T \\ \vdots \\ Y_n^T \end{bmatrix}$$

Учитывая, что при заданных из выборки значениях X и Y, первая матрица в левой части и матрица в правой части не содержат неизвестных, с помощью МНК получим неизвестные значения элементов матрицы G и вектор-столбца h, т.е. коэффициенты функций принадлежности выходной переменной

Неизвестные значения элементов матрицы *G* и вектор-столбца *h*, т.е. коэффициенты заключений правил находятся с помощью МНК

1.4.7. Настройка параметров нейро-нечёткой системы

На данном этапе производится настройка параметров модели с помощью методов обучения нейронных сетей. Настройка производится на основе метода обратного распространения ошибки. Для использования методов обучения нейронных сетей для настройки нечёткой модели типа Сугено-Такаги представим её в виде эквивалентной ей нейронной сети прямого распространения следующего вида, как показано на рис.

С учётом приведённого в п.1.4.2 описания структуры системы в процессе обучения будут настраиваться параметры нейронов в слое 1 и слое 4.

Настройка нейронов слоя 1, по сути, является настройкой функций принадлежности (параметров сдвига и масштаба) термов входных переменных, а слоя 4 – коэффициентов в выражениях заключений решающих правил.

Процесс обучения сети состоит из прямого и обратного проходов на каждой эпохе. Во время прямого прохода мы предъявляем выборку на вход сети и функциональные сигналы идут по ней в прямом направлении, чтобы рассчитать выход каждого узла сети. После этого определяется мера ошибки относительно данных из обучающей выборки и производная:

$$E_p = \left(y_{T,p} - y_{0,p}\right)^2 ,$$

где p – номер набора данных из выборки T, $y_{T,p}$ - значение выходной переменной из выборки, $y_{0,p}$ - значение выходной переменной, рассчитанное в ходе прямого прохода по сети.

$$\frac{\partial E_p}{\partial y_{0,p}} = -2\left(y_{T,p} - y_{0,p}\right)$$

Во время обратного прохода по сети последовательно идёт расчёт производной меры ошибки на выходе каждого из внутренних слоёв,

$$\frac{\partial E_p}{\partial O_{i,p}^k} = \sum_{m=1}^{N_{k+1}} \frac{\partial E_p}{\partial O_{m,p}^{k+1}} \frac{\partial O_{m,p}^{k+1}}{\partial O_{i,p}^k},$$

где $O_{i,p}^k$ - значение на выходе i-ого узла k-того слоя при предъявлении на вход сети p-ого набора данных из выборки, N_{k+1} - количество узлов в $k+1$ слое.

Если обозначить корректируемый параметр узла сети α, тогда получим

$$\frac{\partial E_p}{\partial \alpha} = \sum_{O^* \in S} \frac{\partial E_p}{\partial O^*} \frac{\partial O^*}{\partial \alpha}$$

где S – множество всех узлов, выходное значение которых зависит от α. Тогда значение производной величины ошибки для всей эпохи будет следующим:

$$\frac{\partial E}{\partial \alpha} = \sum_{p=1}^{P} \frac{\partial E_p}{\partial \alpha},$$

где P – объём обучающей выборки.

Соответственно, параметры 1-ого слоя (параметры функций принадлежности) и 4-ого слоя (параметры коэффициентов заключений нечетких правил) корректируются согласно методу градиента по следующей формуле:

$$\Delta\alpha = -\eta \frac{\partial E}{\partial \alpha}$$

где η – коэффициент шага коррекции, $0 < \eta \le 1$

Обучение продолжается до тех пор, пока ошибка обучения превышает допустимое значение:

$$\max\left\{\left|y_{T,p} - y_{0,p}\right|\right\} < \varepsilon_{доп}$$

где p – номер набора данных из выборки T, $y_{T,p}$ - значение выходной переменной из выборки, $y_{0,p}$ - значение выходной переменной, рассчитанное в ходе прямого прохода по сети.

1.2.4. Нейро-нечёткие системы в задачах управления

Применение нейро-нечётких систем в задачах управления определяется двойственностью их представления: с одной стороны как нечётких систем вида Сугено-Такаги, а с другой стороны, как ИНС специального вида. Поэтому регуляторы на их основе могут настраиваться экспертами, аналогично нечётким регуляторам, а в дальнейшем донастраиваться на основе поступающих в ходе эксплуатации данных.

Другим возможным подходом является построение подражающего нейро-нечёткого регулятора. При этом после настройки заключением каждого из правил такого регулятора может быть интерпретировано в виде закона управления для определённых значений входных параметров.

2. ОПИСАНИЯ ЛАБОРАТОРНЫХ РАБОТ

Лабораторная работа №1. Изучение основ применения искусственных нейронных сетей

Цель работы – изучение основ работы с искусственными нейронными сетями (ИНС) в среде ППП Simintech.

Одно из возможных применений ИНС, которое рассматривается в курсе – оценка показателей надёжности автоматизированных систем.

Рассмотрим в качестве объекта исследования элемент автоматизированной информационно-измерительной системы коммерческого учёта электроэнергии (АИИС КУЭ) – информационно-вычислительный комплекс электроустановки (ИВКЭ).

Основу информационно-измерительного канала (ИИК) системы составляют измерительные трансформаторы тока (ТТ), измерительные трансформаторы напряжения (ТН), их вторичные цепи, трёхфазный счетчик электрической энергии. Таким образом, измерительные трансформаторы тока и напряжения, входящие в состав системы, предназначены для преобразования большого тока и высокого

напряжения сети к уровням, соответствующим входным токам и напряжениям счётчиков электрической энергии, иными словами, они являются масштабирующими устройствами.

Следующим элементом системы, образующим вместе с ТТ и ТН ИИК, является микропроцессорный счётчик электрической энергии. Он предназначен для измерения перетоков активной и реактивной электрической энергии и мощности. Вдобавок, он способен выполнять интегрирование результатов измерений на получасовых интервалах и сохранение полученных значений в памяти с привязкой к текущему времени (профили нагрузки).

На уровне ИВКЭ происходит сбор и объединение результатов измерений по отдельным ИИК в групповые измерения по НПС и подготовка информации для передачи на более высокие уровни АИИС КУЭ. Обычно данные функции выполняет комплексное устройство учёта и автоматизации (КУУиА).

Учитывая нормативные документы, применяемые в области эксплуатации АИИС КУЭ, и тот факт, что как АИИС КУЭ, так и ИВКЭ являются восстанавливаемыми системами, то в качестве показателя, характеризующего её состояние, используется коэффициент готовности.

Исходя из анализа структуры ИВКЭ, для использования для оценивания надёжности в качестве входных переменных можно использовать следующие.

Срок эксплуатации компонентов, входящих в ИВКЭ.

Выбор данной переменой связан с тем, что на различных этапах жизненного цикла ИВКЭ показатели надёжности отдельных устройств (например, интенсивность отказов), входящих в него могут принимать не только различные значения, но и менять вид закона распределения (последний фактор часто не учитывается при расчётах обычными методами).

Сложность ИВКЭ предлагается измерять *количеством установленных счётчиков электрической энергии*.

Выбор данного показателя связан с тем, что, с точки зрения теории надёжности, все компоненты, входящие в ИВКЭ представляют собой последовательное соединение элементов. Т.е. надёжность такой системы зависит от количества, входящих в неё устройств и, в значительной степени, определяется самым слабым звеном в цепи элементов. Учитывая то, что счётчики электрической энергии являются одними из наиболее сложных с технической точки зрения и многочисленными устройствами, входящих в ИВКЭ, можно предположить, что они как раз и являются этим слабым звеном.

Максимальное по модулю отклонение от номинального режима величины тока в измерительных цепях компонентов ИВКЭ.

Как показывает практика, отклонение режима работы от номинального для любого устройства ведёт к ухудшению его показателей надёжности. Режим работы компонентов ИВКЭ может характеризоваться температурными условиями и влажностью в помещении, напряжением питания, напряжённостью электромагнитного поля и т.п. Однако, как показывают исследования, на практике чаще всего имеет место влияние отклонения величины тока в измерительных цепях ТТ, ТН и счётчиков электрической энергии. При значениях тока ниже номинального происходит рост суммарной погрешности измерений вплоть до выхода за допустимые пределы. А такая ситуация может расцениваться как отказ, т.к. система перестаёт правильно выполнять свои функции. При значениях тока, выше номинального к росту погрешности добавляется повышенная нагрузка на элементы, которая может привести к выходу устройств из строя.

По результатам эксплуатации подобного рода систем, как правило, накапливается значительный объём статистических данных, который можно использовать для построения ИНС, которая будет представлять собой оценку зависимости показателя надёжности от внешних факторов. В результате, с помощью такой ИНС можно оценить значение коэффициента готовности по значениям параметров сложности, срока эксплуатации и отклонения режима эксплуатации.

В лабораторной работе необходимо построить ИНС, позволяющей оценивать надёжность, на основе выборки с тремя параметрами, характеризующими информационно-измерительные системы коммерческого учёта электроэнергии (коэффициент сложности, среднюю загруженность оборудования, срок эксплуатации оборудования), и значением параметра надёжности (коэффициента готовности), соответствующего данным системам.

Порядок выполнения работы.

1. Ознакомьтесь с принципами работы с искусственными нейронными сетями в ППП Simintech на примере проекта классификации ирисов (\SimInTech\Demo\Нейронные сети\Классификация ирисов\Обучение.prt).
2. Изучите состав схемы, назначение блоков и их свойства. Изучите пункт меню «Параметры расчёта» Проведите обучение данной сети. Сделайте вывод о точности обучения.
3. Проведите исследование работы обученной сети, для этого откройте проект (\SimInTech\Demo\Нейронные сети\Классификация ирисов\Тестирование.prt).
4. Исследуйте влияние увеличения и уменьшения количества слоёв на качество обучения.

5. На основе изученных схем создайте свою схему для классификации систем по степени надёжности. Сохраните созданный проект в своей папке на ПК.

6. Данные, необходимые для выполнения работы, находятся в файлах *inputXX.txt, targetXX.txt*. Для загрузки данных из файлов в среду Simintech воспользуйтесь блоком «Таблица данных из файла».

7. При настройке размерности входных данных укажите, что в них содержится 3 параметра.

8. Проведите обучение и тестирование полученной сети, аналогично примеру с ирисами. Сделайте выводы о точности обучения и влиянии количества слоёв и нейронов в слое на качество обучения.

Вопросы к защите:

1. Искусственный нейрон. Структура и принцип работы.
2. Свойства искусственных нейронных сетей (ИНС). Классификация искусственных нейронных сетей.
3. Многослойная (двухслойная) сеть прямого распространения. Принципы работы.
4. Обучение нейронных сетей. Алгоритм обратного распространения ошибки
5. Алгоритм обратного распространения ошибки. Геометрическая интерпретация.
6. Переобучение. Причины возникновения. Способы избегания переобучения.
7. Алгоритмы обучения без учителя. Особенности.
8. ИНС в задачах управления. Приведите пример.
9. ИНС в задачах анализа состояния технических систем. Приведите пример.
10. Опишите процесс работы и настройки нейрорегулятора.

Лабораторная работа №2. Изучение основ применения генетических алгоритмов

Цель работы – изучение основ работы с генетическими алгоритмами.

В работе рассматривается решение задачи поиска минимума функции на заданном интервале с применением генетических алгоритмов.

Порядок выполнения работы.

1. Проанализируйте функцию, соответствующую вашему варианту (табл. 2.1), и постройте её график.

Таблица 2.1

№ вар.	Функция
1.	$f(x) = 2x^2+1$
2.	$f(x) = 2x^3+125\|2x + 10\|$
3.	$f(x) = 4x^4+16$
4.	$f(x)=\sqrt{\|x + 2\|}$
5.	$f(x)=\sqrt{(x + 2)^2 + 3}$
6.	$f(x) = 2x^8+15$
7.	$f(x) = \|0{,}5x^3 + 10\|$
8.	$f(x) = -4x^4+16$
9.	$f(x)=\sqrt{\|x + 2\|}$
10.	$f(x) = 2\|x^7\|+11$

2. Сформулируйте задачу поиска оптимального значения функции на отрезке [0;15] с точностью до целых значений в терминах генетических алгоритмов.

3. Разработайте программу поиска максимального значения, с использованием классического варианта генетического алгоритма. Текст программы занесите в отчёт.

4. Запустите процедуру поиска оптимального значения на заданном отрезке. Проанализируйте полученные результаты, оцените точность полученного результата.

5. Модифицируйте программу поиска: измените количество точек скрещивания, тип селекции (по указанию преподавателя). Проанализируйте полученные результаты.

6. Полученные результаты и выводы занесите в отчёт.

Вопросы к защите:

1. Основные понятия генетических алгоритмов.

2. Генетические алгоритмы и традиционные методы оптимизации. Достоинства и недостатки

3. Генетические операторы. Кодирование параметров задачи в генетическом алгоритме.

4. Опишите работу классического генетического алгоритма.

5. Основная теорема о генетических алгоритмах. Следствия теоремы.

6. Модификации классического генетического алгоритма

7. Генетические алгоритмы и ИНС. Способы совместного использования

Лабораторная работа № 3. Изучение основ применения нечёткой логики

Цель работы – изучение основ работы с элементами нечёткой логики в среде Simintech.

В работе рассматривается система управления с нечётким регулятором. Объектом управления является бак с водой, к которому подходят две трубы: через одну трубу, снабженную краном, вода поступает в бак, через другую – вытекает. Подачу воды в бак можно регулировать краном. Расход воды является неконтролируемым и зависит от уровня воды в баке. Если понимать под выходной (регулируемой) переменной уровень воды, а под регулирующим элементом – кран, то подобный объект регулирования с точки зрения его математического описания является динамическим и существенно нелинейным.

Цель управления в данном случае заключается в установлении на требуемом, изменяющемся во времени уровне воды в баке. В регулятор, обеспечивающий достижение цели управления, должна поступать информация о несоответствии (разности) требуемого и фактического уровней воды, при этом данный регулятор должен вырабатывать управляющий сигнал, подаваемый на регулирующий элемент.

Порядок выполнения работы.

1. 1. Откройте в Simintech модель системы с регулятором воды в баке. Модель находится в папке *\SimInTech\Demo\Автоматика и математика\Нечёткая Логика\Система поддержания уровня воды в баке*. Изучите её состав, проанализируйте назначение и параметры блоков. **Сохраните файл модели с новым названием.**

2. Запустите выполнение модели. Проанализируйте графики, выводимые в блоках *«Временной график».* Занесите их в отчёт.

3. Изучите модель нечёткого регулятора воды в баке, кликнув по ней на схеме. Изучите раздел «Справка» для блоков модели. Изучите переменные регулятора, термы и решающие правила. Занесите в отчёт решающие правила, используемые в регуляторе, а также графики функций принадлежности (*можно построить в сторонней программе по параметрам, указанным в блоках*).

4. Изучите в редакторе состав и параметры нечёткого регулятора, а также используемый тип алгоритма нечёткого вывода. Обратите внимание на возможность настройки способов реализации логических операций «И», «ИЛИ», логического вывода, объединения полученных заключений правил, дефаззификации.

5. Изучите способы создания и удаления термов и функций принадлежности (ФП).

6. Проанализируйте представленный набор правил. Обратите внимание на возможные предпосылки правил (способы задания связок между ними), а также заключения. Изучите процесс создания, редактирования, удаления правил.

7. Модифицируйте исходную систему в соответствии с вариантами, приведёнными ниже.

Вариант 1: Модифицируйте базу решающих правил. Добавьте учёт скорости изменения уровня воды при высоком и низком уровнях воды в баке.

Вариант 2: Модифицируйте базу решающих правил. Удалите два наименее важных (обоснуйте) решающих правила.

Вариант 3: Измените вид функций принадлежности и границы термов входных переменных.

Вариант 4: Измените вид функций принадлежности заключений правил.

Вариант 5: Замените алгоритм нечёткого вывода Мамдани на алгоритм Цукамото. В качестве параметров заключений правил алгоритма Цукамото используйте точки максимума треугольных функций принадлежности заключений правил алгоритма Мамдани.

Сохраните получившуюся модель в отдельном файле.

8. Запустите моделирование системы. Графики параметров работы системы занесите в отчёт. Проанализируйте работу системы. Что изменилось в работе системы? Выводы занесите в отчёт.

9. Исследуйте работу системы с ПД-регулятором в контуре вместо нечёткого. Для этого нужно использовать блок «ПИД-регулятор» на вкладке «Регуляторы». Обратите внимание на знаки сигналов в сумматоре обратной связи. Откорректируйте их с целью обеспечения отрицательной обратной связи. Подберите параметры регулятора, обеспечивающие устойчивый переходный процесс. Сохраните модель в отдельном файле.

Проанализируйте графики, выводимые в блоках *«Временной график»,* и сравните их с полученными для нечёткого регулятора. Занесите их в отчёт.

Вопросы к защите:

1. Какие виды функций принадлежности Вам известны? Проанализируйте их преимущества и недостатки.
2. Опишите основные этапы нечёткого вывода.
3. Алгоритм Мамдани. Особенности.
4. Алгоритм Цукамото. Особенности.
5. Алгоритм Ларсена. Особенности.
6. Алгоритм Такаги-Сугено. Особенности.
7. .Упрощённый алгоритм нечёткого вывода. Особенности.

8. Опишите работу нечёткого регулятора. Приведите структуру нечёткой системы управления.

9. Опишите применение нечётких систем для решения задач диагностики.

Лабораторная работа №4. Изучение основ применения нейро-нечётких систем

Цель работы – изучение основ работы с элементами нейро-нечётких систем в среде SimInTech.

В работе рассматривается система управления с нечётким регулятором (как в л.р. №3). Объектом управления является бак с водой, к которому подходят две трубы: через одну трубу, снабженную краном, вода поступает в бак, через другую – вытекает. Подачу воды в бак можно регулировать краном. Расход воды является неконтролируемым и зависит от уровня воды в баке. Если понимать под выходной (регулируемой) переменной уровень воды, а под регулирующим элементом – кран, то подобный объект регулирования с точки зрения его математического описания является динамическим и существенно нелинейным.

Необходимо с помощью выборок с записями сигналов управления настройить нейро-нечёткую систему управления.

Порядок выполнения работы.

1. Данные, необходимые для выполнения работы, находятся в файлах E_data_XX.txt и dE_data_XX.txt, где XX – номер варианта.Откройте в среде SimIntech проект «Нечёткая кластеризация». Изучите его содержимое, свойства блоков. **Сохраните его под новым именем.**

3. Проведите кластеризацию данных из файлов с исходными данными с целью определения параметров для создания нейро-нечёткой системы на их основе. Для кластеризации используйте настройки по умолчанию.Зафиксируйте в отчёте параметры, определённые при кластеризации, необходимые для создания нейро-нечёткого регулятора.

5. Откройте проект с регулятором из л.р.3. **Сохраните проект под новым названием.** Запустите моделирование системы. Зафиксируйте график переходного процесса.

6. Модифицируйте нечёткий регулятор с целью его преобразования к системе нейро-нечёткого вида. Настройте его в соответствии с данными проведённой кластеризации.

7. Запустите моделирование получившейся системы. Сравните с исходным вариантом. Сделайте предположения о причинах произошедших изменений.

8. Повторите п.3-7 с новыми параметрами метода кластеризации (табл. 2.2)

Таблица 2.2

Параметры для метода кластеризации

№ варианта	Диапазон влияния
1	0,7
2	0,4
3	0,8
4	0,3
5	0,55
6	0,35
7	0,6
8	0,45
9	0,75

9. Проведите сравнение получившихся в п.4 и п.6 систем. Как повлияло изменение параметров на количество решающих правил, форму функций принадлежности, точность работы системы. Результаты занесите в отчёт.

Вопросы к защите:

1. Нейро-нечёткие системы. Предпосылки возникновения
2. Нейро-нечёткие системы. Структура системы.
3. Нейро-нечёткие системы. Принципы работы.
4. Нейро-нечёткие системы. Сравнение с нечёткими и нейросистемами.
5. Создание функций принадлежности на основе выборки данных.
6. Использование методов ИНС для обучения нейро-нечётких систем.
7. Модификации нейро-нечётких систем. Особенности.
8. Нейро-нечёткие системы в задачах управления. Приведите пример.
9. Нейро-нечёткие системы в задачах диагностики. Приведите пример.

СПИСОК РЕКОМЕНДОВАННОЙ ЛИТЕРАТУРЫ

Основной

1. Борисов В.В., Круглов В.В., Федулов А.С. Нечеткие модели и сети. – 2-е изд., стереотип. – М.: Горячая линия–Телеком, 2018. – 284 с.

Дополнительный

2. Рутковская Д., Пилиньский М., Рутковский Л. Нейронные сети, генетические алгоритмы и нечеткие системы: Пер. с польск. И.Д. Рудинского. – 2-е изд., стереотип. – М.: Горячая Линия–Телеком, 2013. – 452 с.:ил.

3. Пегат А. Нечеткое моделирование и управление: пер. с англ. – 2-е изд.–М.: БИНОМ. Лаборатория знаний, 2017.–798 с.: ил.